

Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Sistemas Operativos I

Trabajo Práctico Final

Integrantes:

Lautaro Capezio
Luciano Duarte

Índice

1. Introducción	2
2. Estructura del nodo y organización por procesos	2
3. Diseño de concurrencia	2
3.1. Modelo adoptado	2
3.2. Justificación de las decisiones clave	3
3.3. Exclusión mutua	3
3.4. Sincronización interna	4
3.5. Deadlocks y bloqueos	4
4. Robustez	5
4.1. Errores manejados	5
4.2. Estrategias de recuperación	6
5. Comparación de paradigmas	6
5.1. Qué simplificó Erlang	6
5.2. Qué complejizó Erlang	7
5.3. Ventajas de C para este problema	7
5.4. Desventajas de C para este problema	7
6. Conclusión	9

1. Introducción

El trabajo implementa un sistema P2P descentralizado para una red local en el que cada nodo cumple simultaneamente el rol de servidor y cliente. El sistema utiliza UDP para descubrimiento y consenso de identificadores únicos, y TCP para búsqueda, verificación de integridad y transferencia de archivos. Además, soporta descargas paralelas por chunks desde múltiples fuentes.

La implementación se realizó en Erlang, por lo que el diseño de concurrencia se apoya en procesos livianos y paso de mensajes. Esta decisión afecta de forma directa la estructura del programa, la forma de encapsular el estado, la estrategia de sincronización y la manera de tolerar fallas parciales.

2. Estructura del nodo y organización por procesos

El nodo se organiza alrededor de un proceso coordinador principal, implementado en `p2p_node.erl`, que inicia y detiene el resto de los componentes. A partir de ese coordinador se levantan procesos especializados:

- `file_manager`: mantiene la vista local de la carpeta compartida y atiende consultas sobre archivos.
- `node_registry`: conserva el conjunto de nodos conocidos y depura entradas inactivas.
- `discovery`: escucha mensajes UDP, participa del consenso de identificadores y registra nuevos nodos.
- `hello_broadcast`: envia periódicamente mensajes HELLO para anunciar presencia.
- `tcp_server`: acepta conexiones TCP y crea un proceso independiente por cliente.
- `download_manager`: coordina el estado de una descarga multi-fuente y asigna chunks a workers.
- `cli`: ejecuta la interfaz interactiva sin bloquear al coordinador.

La descomposición por procesos cumple un doble objetivo. Por un lado, separa responsabilidades funcionales; por otro, evita compartir memoria mutable entre módulos concurrentes. Cada componente posee su propio loop de recepción y su propio estado local.

3. Diseño de concurrencia

3.1. Modelo adoptado

El modelo de concurrencia de Erlang se basa en procesos livianos aislados, cada uno con heap propio y comunicación mediante envío asincrónico de mensajes. En esta aplicación ese enfoque aparece de manera explícita:

- El coordinador del nodo se crea con `spawn` y conserva referencias a procesos hijos en su diccionario de proceso.
- `node_registry` mantiene su estado como un mapa interno y lo modifica solo dentro de su loop.

- `download_manager` centraliza el estado de cada descarga multi-fuente y responde pedidos de asignación, progreso y finalización.
- `tcp_server` acepta conexiones y crea un proceso por cliente con `spawn_link`, permitiendo atención concurrente de múltiples pedidos.
- Las búsquedas distribuidas lanzan un proceso por nodo remoto para consultar en paralelo.
- Las descargas multi-fuente lanzan varios workers por nodo, de modo que diferentes chunks puedan descargarse simultáneamente.

Esta estrategia reemplaza un diseño clásico con hilos compartiendo estructuras globales. En vez de tener tablas compartidas protegidas por mutexes, el estado queda encapsulado en procesos propietarios.

3.2. Justificación de las decisiones clave

La primera decisión importante fue utilizar un proceso coordinador principal para el ciclo de vida del nodo. Esto simplifica el arranque y apagado ordenado, porque un único proceso conoce los PIDs relevantes y puede cerrar CLI, servidor TCP, registro, descubrimiento y broadcaster de manera consistente.

La segunda decisión clave fue modelar los componentes con estado como servidores manuales basados en `receive`. En particular, `node_registry` y `download_manager` funcionan como monitores serializadores: reciben mensajes, actualizan su estado y vuelven al loop. De esa forma se evita que dos tareas modifiquen simultáneamente la misma estructura.

La tercera decisión relevante fue paralelizar solo el trabajo que no requiere estado compartido complejo. Por ejemplo, el servidor TCP crea un proceso por conexión porque cada cliente maneja su propio socket; la búsqueda distribuida crea un proceso por nodo remoto porque cada consulta es independiente; la descarga multi-fuente crea workers concurrentes, pero las decisiones de asignación de chunks quedan centralizadas en `download_manager`.

3.3. Exclusión mutua

En esta implementación, la exclusión mutua no se logra con mutexes sino con **propiedad exclusiva del estado**. El patrón es el siguiente:

- Un proceso es el único autorizado a modificar cierta estructura.
- Los demás procesos no acceden directamente al estado, sino que envían pedidos.
- El proceso propietario atiende esos pedidos secuencialmente en su mailbox.

Este esquema aparece claramente en dos módulos:

- `node_registry`: el mapa de nodos conocidos solo se modifica dentro de `loop/1`. Cuando llega `{add_node, ...}`, el proceso actualiza `last_seen`, elimina entradas obsoletas con misma IP:puerto y conserva una vista consistente.
- `download_manager`: el estado de chunks (`pending`, `downloading`, `complete`) y las asignaciones activas solo cambian dentro de un único loop. Así se evita que dos workers reciban el mismo chunk a la vez.

Desde el punto de vista conceptual, el mailbox del proceso cumple el rol que en C cumpliría una sección crítica protegida por mutex. La diferencia es que en Erlang la serialización está integrada al modelo y no requiere bloqueos explícitos alrededor de estructuras compartidas.

3.4. Sincronización interna

La sincronización entre componentes se resuelve con mensajes y timeouts. Algunos ejemplos relevantes:

- **Arranque del nodo:** `p2p_node:start/1` espera el mensaje `{node_started, Pid}` del coordinador antes de quedar bloqueado hasta el apagado.
- **Consultas al registro:** funciones como `get_all_nodes/0` y `get_node/1` envían una solicitud y esperan una respuesta puntual del proceso `node_registry`.
- **Asignación de chunks:** cada worker llama a `download_manager:assign_chunk/2`; el coordinador de descargas responde con un chunk disponible o con `no_chunks`.
- **búsqueda distribuida:** el proceso invocante lanza consultas paralelas y luego consolida respuestas con `collect_results`.
- **Descarga multi-fuente:** el proceso principal espera eventos `worker_done` y `worker_error` hasta que todos los chunks estén completos o se alcance un timeout global.

En todos estos casos, la sincronización es cooperativa y basada en protocolos de mensajes simples. No hay variables de condición ni semáforos expuestos al programador.

3.5. Deadlocks y bloqueos

En una implementación equivalente en C con hilos, este problema sería especialmente sensible a deadlocks por adquisición de múltiples mutexes en distinto orden, olvido de liberar locks, o esperas condicionales mal coordinadas. En esta solución, ese tipo de deadlock se reduce drásticamente porque no hay locks explícitos ni memoria compartida mutable entre procesos.

Sin embargo, Erlang no elimina todos los riesgos de bloqueo. Los principales riesgos pasan a ser:

- **Esperas indefinidas por respuesta:** un proceso puede quedar esperando un mensaje que nunca llega si el emisor falla o no responde.
- **Timeouts de red:** una operación TCP o UDP puede quedar demorada por desconexión, nodo caído o indisponibilidad del socket.
- **Starvation lógica:** un chunk podría quedar sin completar si no existiera una política de reasignación tras un fallo.

Estos riesgos se mitigaron de varias maneras:

- La mayoría de las operaciones sincrónicas sobre procesos internos usan `after` con timeouts, por ejemplo en `node_registry`, `download_manager` y `search`.

- Las operaciones TCP usan timeouts configurables para conexión, lectura de cabecera y recepción de chunks.
- Cuando un worker falla al bajar un chunk, el coordinador lo vuelve a marcar como `pending`, habilitando el reintento por otro worker o por la misma fuente.
- Los workers pueden dejar de insistir con una fuente cuando detectan errores terminales como `node_not_found`, `connection_refused` o `connection_timeout`.

Por lo tanto, el diseño evita deadlocks clásicos de exclusion mutua, pero aun requiere manejo explícito de bloqueos logicos y de fallas en la comunicación.

4. Robustez

4.1. Errores manejados

La robustez del sistema descansa en una combinacion de deteccion temprana de errores, timeouts y recuperación por reintento o degradacion controlada. Los casos cubiertos mas importantes son los siguientes:

- **Fallo al abrir sockets UDP:** durante la solicitud de ID, si `gen_udp:open` falla, el modulo de descubrimiento informa el error, espera un segundo y reintenta.
- **colisión de identificadores:** si otro nodo rechaza un ID propuesto mediante `INVALID_NAME`, se espera un tiempo aleatorio y se vuelve a solicitar un identificador.
- **Desconexión o inactividad de nodos:** `node_registry` elimina nodos cuyo `last_seen` supera el umbral de inactividad configurado.
- **Errores de aceptacion TCP:** `tcp_server` captura errores en `accept` y continúa escuchando para no derribar el servicio.
- **Excepciones en handlers de clientes:** cada conexión se procesa con `handle_client_safe/1`, que captura excepciones, registra el error y cierra el socket.
- **Timeouts de lectura:** el servidor y los clientes TCP usan timeouts para evitar quedar bloqueados indefinidamente esperando datos.
- **Archivo inexistente:** tanto para descarga completa como para chunks, si el archivo o fragmento no existe se responde con el código correspondiente de `not found`.
- **Errores de integridad:** la descarga simple valida `sha256` del archivo completo; la descarga multi-fuente reensambla el archivo y compara el hash con el informado por alguna fuente.
- **Falla en descarga de chunk:** si una descarga parcial falla, el chunk vuelve a estado `pending` para que pueda ser reasignado.
- **Fuente no confiable o caída:** el worker puede abandonarla cuando el error indica que insistir no tiene sentido.

4.2. Estrategias de recuperación

No todos los errores se tratan igual. La estrategia cambia segun si el error es transitorio, definitivo o si afecta unicamente a una sub-tarea concurrente:

- **Reintento local:** se usa al abrir el socket UDP del consenso de ID.
- **Reasignacion de trabajo:** se usa en descargas por chunks; un fallo no invalida toda la descarga sino solo la unidad de trabajo afectada.
- **Aislamiento por proceso:** si falla el handler de una conexión TCP, no cae el servidor completo.
- **Degradacion controlada:** si no se puede obtener el checksum remoto, la descarga puede completarse igual, pero se informa que no se pudo verificar integridad remota.
- **Depuracion periódica:** el registro de nodos remueve entradas inactivas para evitar decisiones basadas en fuentes obsoletas.
- **Apagado ordenado:** el coordinador principal cierra procesos auxiliares para no dejar recursos o loops activos al salir.

5. Comparación de paradigmas

5.1. Qué simplificó Erlang

Para este tipo de aplicacion distribuida y concurrente, Erlang simplificó varias cuestiones estructurales:

- **Concurrencia mas barata:** lanzar un proceso por conexión, por consulta o por worker es natural y de bajo costo.
- **Aislamiento del estado:** al no compartir memoria, disminuye mucho el riesgo de corrupcion de estructuras y de carreras de datos.
- **sincronización mas clara:** el modelo pedido-respuesta sobre mailboxes es facil de razonar y encaja bien con servicios como registro de nodos y gestor de descargas.
- **Tolerancia a fallas localizada:** una excepcion en un handler TCP afecta a esa conexión y no necesariamente al resto del sistema.
- **comunicación distribuida natural:** la semantica de procesos y mensajes se alinea muy bien con un sistema P2P, donde gran parte de la lógica ya consiste en intercambio de mensajes.

En C, lograr el mismo nivel de paralelismo habría requerido threads o un esquema de `select/poll/epoll`, junto con una capa manual para coordinacion y proteccion de estructuras compartidas. Eso suele aumentar la complejidad accidental del código.

5.2. Qué complejizó Erlang

También hubo costos y complejidades específicas del paradigma:

- **Mayor necesidad de diseñar protocolos:** al no haber acceso directo a memoria compartida, incluso consultas simples requieren definir mensajes, formatos y respuestas.
- **sincronización explícita por mensajes:** cuando un proceso necesita una respuesta, hay que gestionar `receive`, correlación de mensajes y timeouts.
- **Control más manual fuera de OTP:** al implementar servidores con `spawn` y `receive` en lugar de `gen_server`, parte de la robustez queda en manos del desarrollador.
- **Depuración de flujos concurrentes:** aunque se evitan data races, sigue siendo necesario rastrear interacciones entre procesos y orden de mensajes.

Es decir, Erlang simplifica la seguridad de concurrencia de bajo nivel, pero exige mayor disciplina en el diseño de protocolos entre actores.

5.3. Ventajas de C para este problema

Un enfoque en C habría tenido algunas ventajas puntuales:

- Mayor control fino sobre memoria, buffers y uso del sistema operativo.
- Posible mejor rendimiento bruto en ciertas rutas muy optimizadas de E/S o copiado de datos.
- Mayor cercanía con primitivas POSIX típicas de la materia, como threads, mutexes y variables de condición.

Sin embargo, esas ventajas vienen acompañadas por una carga de implementación significativamente mayor para una aplicación de este tipo.

5.4. Desventajas de C para este problema

En un sistema P2P con descubrimiento, múltiples conexiones simultáneas, workers de descarga y estado compartido, C habría agregado complejidades importantes:

- Necesidad de proteger estructuras globales como el registro de nodos o el estado de descarga mediante mutexes.
- Riesgo real de carreras de datos, deadlocks y errores por orden incorrecto de locking.
- Mayor dificultad para aislar fallas de una conexión individual sin comprometer threads o estructuras compartidas.
- Mayor trabajo manual para manejo seguro de memoria, buffers y limpieza de recursos ante errores intermedios.

Para una aplicacion distribuida con fuerte componente concurrente, estas desventajas son especialmente relevantes.

Para este trabajo, el paradigma de Erlang fue particularmente adecuado porque el problema esta dominado por coordinacion entre actores concurrentes y comunicaci3n en red, no por calculo intensivo local. En consecuencia, el modelo de procesos y mensajes simplific3 la arquitectura general y redujo errores tipicos de concurrencia compartida.

6. Conclusión

El sistema implementado muestra una correspondencia fuerte entre el dominio del problema y el paradigma elegido. Al tratarse de un nodo P2P con descubrimiento, consultas remotas, transferencia de archivos y descargas paralelas, el modelo de Erlang permitio estructurar la solucion como un conjunto de procesos cooperativos, cada uno responsable de un fragmento bien definido del estado.

La exclusion mutua se resolvio sin mutexes mediante propiedad exclusiva del estado y serializacion por mailbox. La sincronización se realizó mediante mensajes y timeouts. Los deadlocks clásicos del modelo de hilos se evitaron, aunque siguio siendo necesario tratar bloqueos logicos y fallas de comunicación. En robustez, el sistema incorpora reintentos, deteccion de nodos inactivos, aislamiento de errores por conexión, validacion de integridad y reasignacion de chunks fallidos.

En Comparación con una solucion en C, Erlang reduce la complejidad accidental de la concurrencia y mejora la claridad del diseño distribuido, a costa de requerir mas trabajo en la definicion de protocolos de mensajes y, en esta version, de no aprovechar completamente herramientas OTP como supervisores y comportamientos estandar. Aun asi, para la naturaleza de esta aplicacion, la eleccion de Erlang resulto ventajosa y coherente con los objetivos del trabajo.